

Lab 3 Exam

题目中已经帮我们实现好了全部的定义，将题目中给出的代码复制好后，我们就只需要实现两个内容：一是仿照 `env_create()` 函数补全 `env_create_edf()` 函数；二是根据代码中的注释修改 `schedule()` 函数。

我们首先进行第一步，将 `env_create()` 函数中为新进程的属性赋值部分替换为题目中给出的内容，再将插入 `env_sched_list` 队列改为插入 `env_edf_sched_list` 队列即可（这里需要注意两个队列的类型是不同的，插入时要改用 `LIST_INSERT_HEAD`）

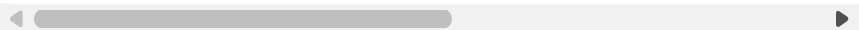
```
// kern/env.c
struct Env *env_create_edf(const void *binary, size_t size)
{
    struct Env *e;

    env_alloc(&e, 0);

    e->env_edf_runtime = runtime;
    e->env_edf_period = period;
    e->env_period_deadline = 0; // 初始化为 0，使进程在首
    e->env_status = ENV_RUNNABLE;

    load_icode(e, binary, size);
    LIST_INSERT_HEAD(&env_edf_sched_list, e, env_edf_sched_list)

    return e;
}
```



接下来就是本次 exam 的重难点，即修改 `schedule()` 函数。在动手操作之前，请一定确保自己完全了解了题目中的调度规则。如果有疑惑的话，可以参考一下题目描述中的“示例”一节，我个人觉得这一节的讲解是最清楚的。

接下来我们跟着注释一步一步完成代码：

(1) 遍历 `env_edf_sched_list`，如果进程进入了新的运行周期（可通过 `clock == env_period_deadline` 判断），则更新 `env_period_deadline`，并将 `env_runtime_left` 设置为 `env_edf_runtime`。

关于如何更新 `env_period_deadline`，从题目描述的示例可以总结出，`env_period_deadline` 是 `env_edf_period` 的倍数，每当进程到达截止时间时，将截止时间再加上一倍即可：

```
// kern/sched.c
struct Env *env; // 循环变量

LIST_FOREACH(env, &env_edf_sched_list, env_edf_sched_list)
    // 这里可以根据提示，使用 LIST_FOREACH 来循环
    if (clock == env->env_period_deadline) {
```

```

    env->env_period_deadline += env->env_edf_perioc
    env->env_runtime_left = env->env_edf_runtime;
}
}

```

(2) 遍历 `env_edf_sched_list`，选取 `env_runtime_left` 大于 0 且 `env_period_deadline` 最小的进程调度（若相同，则选择 `env_id` 最小的进程）。如果不存在这样的进程，则不进行调度。

首先，我们需要通过遍历找出需要调度的 edf 进程。下一步，我们还需要判断进程是否存在。若存在，我们需要调用 `env_run()` 来运行该进程，**同时注意维护 `env_runtime_left` 变量的值，减去一个时间片**：

```

// kern/sched.c
u_int min = -1; // 截止时间最小的进程的截止时间
struct Env *wheremin = NULL; // 截止时间最小的进程

LIST_FOREACH (env, &env_edf_sched_list, env_edf_sched_l
    if (env->env_runtime_left > 0) { // 这里注意判断当前
        if (min == -1) {
            min = env->env_period_deadline;
            wheremin = env;
        }
        else {
            if (env->env_period_deadline < min || (env-
                min = env->env_period_deadline;
                wheremin = env;
            )
        }
    }
}

if (wheremin != NULL && wheremin->env_status == ENV_RUN
    // 不是很确定是否要判断 status 是否为 ENV_RUNNABLE ,
    wheremin->env_runtime_left--;
    env_run(wheremin);
}

```

(3) 使用课下实现的 RR 算法调度 `env_sched_list` 中的进程。

到这一步有些同学就看不太懂了，不知道有什么可改的地方，其实有两处是要修改的，当然这就需要我们对本题的调度规则有比较清晰的理解了。

第一处，不知道你有没有怀疑过注释 (3) 的表述，根据上面的规则和示例，显然只有在 edf 队列中没有找到要调度的 edf 进程时，才会从 RR 队列中选取进程调度，也就是说 (3) 中的调度不是无条件的，这就需要将整个 RR 调度的逻辑放在上一步的分支判断中。

第二处，根据代码中的注释和后文的提示都可以看出，我们需要将 RR 调度中的 `e` 由当前进程 `curenv` 改为上次执行的 RR 进程。这里我们需要新开一个 static 变量 `struct Env *lastRRenv`，每次进行 RR 调度时将当前进程存储进去即可：

```
// kern/sched.c
if (wheremin != NULL && wheremin->env_status == ENV_RUN
    wheremin->env_runtime_left--;
    env_run(wheremin);
}
else {
    static int count = 0;
    static struct Env *lastRRenv = NULL;
    // 这里必须为静态变量 static，否则每次调用 schedule()
    // 仿照 curenv 的定义，初值赋为 NULL
    // struct Env *e = curenv;
    struct Env *e = lastRRenv; // 删去 e 的原定义，改为：

    if (yield || count == 0 || e == NULL || e->env_stat
        if (e != NULL && e->env_status == ENV_RUNNABLE)
            TAILQ_REMOVE(&env_sched_list, e, env_sched_
            TAILQ_INSERT_TAIL(&env_sched_list, e, env_s
        }
        e = TAILQ_FIRST(&env_sched_list);
        lastRRenv = e; // 此时新的 e 已经确定，更新 lastR
        if (e == NULL) {
            panic("schedule: no runnable envs\n");
        }
        count = e->env_pri;
    }
    count--;
    env_run(e);
}
```

到这里我们就将本次 exam 全部解决了！总的来说，本次 exam 还是非常吃细节的，只有将题目中的调度规则完全搞清楚，并且在做题时不断思考实现细节，时刻记得维护属性值，才能顺利通过测试。

Lab 3 Extra

事实上，Lab 3 extra 的题目虽然总是异常处理，但和 CO P7 的关系真的不是特别大，毕竟还要照顾一下 CO 没学过异常的小伙伴们。其实只要对 MIPS 指令结构和作用有稍微的了解，就足以拿下这道问题了。

异常的注册在提示部分就已经全部给出了，我们只需要实现 `do_adel()` 和 `do_ades()` 两个函数即可。其实只要通读一遍题目，稍作思考就可以发现，这两个函数是一模一样的，除了最后的输出以外没有任何区别。

题目中没有给出具体的实现步骤，现在我们就来拆解一下函数的实现流程：首先我们需要取出当前出现异常的指令，然后对指令进行修改，最后将指令写回原来的位置。接下来我们一步一步实现上面的流程：

第一步其实是最难的一步，很多没看过往年博客的同学直接被硬控到死，到最后家门都没出去。而机智如我发现往年总这么考，直接把这段代码抄在纸上，考试时无脑爆抄，直接助我拿下全考场手刹，所以同学们知道该怎么做了吧（笑）

↑以防大家不知道，再次提醒一下，这门课是开卷考试，可以携带纸质资料入场，下回记得带~

题目的提示部分提过，我们从 `tf->cp0_epc` 中获取到的地址并不是存放当前指令的真实地址，而是它的虚拟地址。我们需要通过一顿猛如虎的操作将其转化为物理地址，才能将当前指令取出。如果你不能理解下面的代码，记住抄下来就对了！祖传代码，屡试不爽：

```
// kern/traps.c
u_int badva = tf->cp0_epc; // 获取虚拟地址
Pde *pgdir = curenv->env_pgdir; // 获取页目录的基地址
struct Page *p = page_lookup(pgdir, badva, NULL); // 查
u_int badpa = page2pa(p) + (badva & 0xfff); // 将页转换
u_int *badk0va = 0x80000000 | badpa; // 转换为 kseg0 的
u_int code = *badk0va; // code 的值为 32 位 MIPS 指令的
```

接下来，我们就需要按照题目中的要求来更改指令了。这一步比较简单，但也有几个需要注意的地方：

首先我们需要注意，这里的修改绝不是直接将当前指令的 `offset` 低 2 位置 0。回忆 `lw` 和 `sw` 指令的用法，我们可以发现，需要能被 4 整除的并不是最后的 `offset` 值，而是 `GPR[base] + offset` 的结果。所以我们需要先计算出这个值，然后根据它对 4 取模的结果调整 `offset`。

于是就出现了另外一个问题，就是在计算 `GPR[base]` 的时候，该如何从寄存器中取值。其实只需要用一步 `int gpr_base = (int)tf->regs[base];` 即可取到，但这一步或许也难住了许多英雄汉，属实可惜。

最后就是要时刻注意其中不断变化的数据长度和有无符号问题了。整个代码中涉及到 `int` `u_int` `signed short` 类型的反复转换，虽然写法多种多样（比如我就是乱写的，我也不知道哪些有用哪些没用），但还是要尽量避免类型的隐式转换，让整个计算过程都在我们的可控范围之内。同时也一定要格外注意长度转换带来的有无符号拓展问题，确保 `offset` 在计算时将 `signed short` 类型转换为 `int` 类型：

```
// kern/traps.c
u_int base = ((code & 0x03e00000) >> 21); // 取出指令的
u_int u_offset = (code & 0x0000ffff); // 取出指令的 0 ~

int gpr_base = (int)tf->regs[base]; // 从寄存器中取出基址
signed short offset = (signed short)u_offset; // 我也不
int now_res = gpr_base + (int)offset; // 算出 GPR[base]
int fix_res = (now_res & ~0x3); // 将上述值的低 2 位置 0
int sub = now_res - fix_res; // 算出置 0 前后的差值
offset = offset - (signed short)sub; // 在 offset 上减去
code = ((code & 0xffff0000) | ((u_int)offset & 0x0000ff
```

最后一步就是把修复好的指令放回原位置了，因为我们刚才已经算出指令在 kseg0 的虚拟地址了，怎么把它取出来就怎么把它放回去即可。最后别忘了按照题目要求输出：

```
// kern/tras.c
*badk0va = code;

// in do_adel():
printk("AdEL handled, new imm is : %04x\n", code & 0xff

// in do_ades():
printk("AdES handled, new imm is : %04x\n", code & 0xff
```

最后还是总结一下，毕竟是 extra，难度还是有一些的，我觉得涉及到地址转换的题目都还挺难的。不过把上面那段八股抄下来直接可秒，后面难度就不太大了，可能涉及到一些位运算的知识，不过我相信大家都能 hold 住。总而言之，多看看往年题，多看看往届博客，好处还是大大的有。大家认真准备，千万不要裸考口牙！